Finding the Memory Dependences Between GPU Kernels by Interpreting the Program's CUDA Source

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A CUDA program runs as a sequence of operations on the GPU: kernel launches, copies between host and device memory, and memory fills. One operation depends on an earlier one when it reads bytes the earlier one wrote, or writes bytes the earlier one read or wrote. A runtime system that knows these dependences can run independent operations at the same time or reorder them, but in graph programs and other irregular code the bytes a kernel touches are chosen by values in memory, such as the neighbour lists of a graph, so a compiler cannot determine them from the source code. We present two interpreters that determine them before the program runs on the GPU, from the program's CUDA source and its input. The whole-program interpreter runs host code and kernels from source on the CPU. The hybrid interpreter runs the host code natively, compiled against a stand-in CUDA runtime whose device memory is host memory, and interprets each kernel on the real memory contents. Both run all threads of a kernel at once as arrays, so a value loaded from memory, a branch on it and an address computed from it are known exactly. When threads load pointers to different buffers, the interpreter splits them into groups by buffer. When the bytes a kernel touches depend on the order in which threads run, the interpreter runs several legal thread orders, reports a dependence as certain only if every order produces it, and adds every dependence any order could produce as possible, generalised over the number of rounds a loop on the host runs. We compare the interpreters against the bytes each operation touched on an NVIDIA A30, recorded with an NVBit tool. On 37 unmodified programs from Rodinia, GraphBIG and PolyBench-GPU, the hybrid interpreter reports none of the GPU's 15,992 dependences wrongly and misses none: 15,852 are reported as certain and 140 as possible, and it reports 1,443 possible dependences that the traced run did not have, all in programs whose behaviour depends on thread order or that read past the end of a buffer. The whole-program interpreter is exact on the 26 programs whose host code it can run, and misses dependences where it cannot, such as host code that reads its input through C++ streams; it reports this. The cost is 9 to 941 seconds per program (median 27) for the hybrid interpreter under four thread orders.

1 Introduction

A CUDA program is a host program, running on the CPU, that issues work to the GPU. The work arrives at the GPU as a sequence of *operations*: a *kernel launch* runs a function, the kernel, on many GPU threads at once, a *copy* moves bytes between host memory and device memory or within device memory, and a *fill* (`cudaMemset`) sets a range of device memory to one value. By default the GPU runs these operations one after another, in the order the host issued them. That order is always safe, but it is often slower than necessary, because two operations that touch different bytes could run at the same time, and an operation whose inputs are ready could start before

an unrelated operation finishes. CUDA lets a program express such freedom with streams and with task graphs [18], and research systems schedule GPU work from a dependence graph between kernels or parts of kernels [2, 1]. All of these need to know which operations depend on which.

An operation *depends* on an earlier one when they touch the same bytes and at least one of them writes them. For regular code, where every address is a simple function of the thread index and the sizes, a compiler can work out the bytes each kernel touches from the source code alone [10]. For irregular code it cannot. In a graph program the addresses a kernel touches are read from memory. For example, a breadth-first search visits `cost[edges[i]]`, where `edges` holds the graph, and it runs that statement only for vertices whose entry in a frontier array is set, an array that the previous kernel wrote. Which bytes the kernel touches is therefore a function of the input graph and of everything the program computed before the launch. Graph benchmark suites for GPUs consist mostly of such kernels [6, 5, 21].

A tool that schedules operations from predicted dependences pays differently for its two kinds of error. A dependence it misses (a *false negative*) lets it run two operations in the wrong order, which must then be detected and undone, an expensive rollback. A dependence it reports that does not occur (a *false positive*) only costs parallelism. The aim is therefore to miss none, and to report as few that do not occur as possible.

In this work we determine these dependences by running the program, with its input, in an interpreter that records the bytes every operation touches. The first interpreter extends MemPrint's interpreter for C and C++ programs [16], which runs a program's syntax tree from `main()` and keeps the values the program stores in memory, so that an address read from memory is known. The second runs the host code natively and interprets only the kernels, which removes the limits of interpreting host code. We add the execution of kernels, the bookkeeping that turns touched bytes into dependences, and the mechanisms that irregular GPU code needs, and we measure the result against a GPU.

In this work, we make the following contributions:

- We describe a whole-program interpreter and a hybrid interpreter that run all threads of a kernel together as arrays, and report for every pair of operations whether they depend on each other, through which buffer, over how many bytes, and whether the dependence is certain or only possible (Section 3).
- We show how to handle a kernel whose threads use different buffers chosen through pointers stored in memory, by splitting the threads into groups by buffer (Section 3.7), and a kernel whose touched bytes depend on the order in which threads run, by running several legal thread orders and generalising what they could do over any number of rounds (Section 3.8).
- We build an NVBit [20] tool that records the bytes each operation touches on a real GPU, and a scoring method that compares two runs even when they issue a different number of operations (Section 4).
- We evaluate both interpreters on 14 test programs and on 38 unmodified benchmarks on an NVIDIA A30. The hybrid interpreter misses no dependence and reports no wrong certain dependence on the 37 benchmarks it completes (Section 5).

2 Background and terms

This section defines the terms the rest of the paper uses. We assume only that the reader knows C.

Threads, blocks and kernels. A kernel is a C++ function marked `__global__`. The host launches it with a *grid* of B *blocks*, each block a group of T *threads*, and writes the launch as `k<<<B,T>>>(...)`. Every thread runs the whole function. A thread tells itself apart from the others through the built-in values `blockIdx` and `threadIdx`, and usually computes from them the element it works on. Threads of one block can share a small, fast memory (`__shared__` arrays) and wait for each other at a barrier (`__syncthreads()`). Threads of different blocks can communicate only through device memory, and the order in which blocks run is not specified. An *atomic* operation, such as `atomicAdd(&x, 1)`, reads and updates one memory location without another thread's update coming in between, and returns the old value.

Operations and buffers. We call every kernel launch, copy and fill an *operation*, and number the operations in the order the host issues them. A *buffer* is one region of device memory obtained from `cudaMalloc` or `cudaMallocManaged`. We measure memory in *units* of 4 bytes, so a dependence through one `int` or `float` is one unit.

Dependences. Operation b depends on an earlier operation a through a unit of a buffer if one of three things holds. In a *read after write* (RAW), a is the last operation before b that wrote the unit, and b reads it, so b needs the value a produced. In a *write after write* (WAW), b writes a unit that a was the last to write, so b must not finish before a, or the final value would be a's. In a *write after read* (WAR), b writes a unit that a read after the last write, so b must not overwrite it before a has read it. We identify a dependence by its source operation, its destination operation, its kind and its buffer, and we record over how many units it passes.

Certain and possible dependences. A dependence is *certain* if the interpreter computed both accesses exactly, and, when it ran the program under several thread orders (Section 3.8), every order produced the dependence. It is *possible* if at least one of the two operations only perhaps touches it, for example because the access sits under a branch on a value the interpreter does not know, or at an address it could not compute. A tool that consumes the dependences can rely on a certain dependence, and must treat a possible one as present to stay safe.

Needed dependences. If c depends on b and b depends on a, then running the operations in an order that respects the first two dependences also respects any dependence of c on a. A scheduler therefore needs only the dependences that are not implied by a path through others. In graph terms these form the *transitive reduction* of the dependence graph, and we call them the *needed* dependences.

Indirect accesses and data-dependent branches. An access is *indirect* if its address is computed from a value loaded from memory, as in `a[b[i]]`. A branch is *data-dependent* if its condition reads memory, as in `if (mask[tid])`. A dependence is *data-dependent* if it passes through units that were touched by an indirect access or under a data-dependent branch. The interpreter marks these from the source code (Section 3.5). They are the dependences a compiler cannot determine, and the ones this paper is about.

Figure 1 shows both kinds in the kernel that expands the frontier in a breadth-first search, written in the style of Rodinia's `bfs` [7]. Which threads pass the test on `mask[tid]`, which elements of `edges` they read, and which elements of `cost` and `updating` they write all depend on the graph and on the previous levels of the search.

```
__global__ void expand(const Node *nodes, const int *edges, bool *mask, bool *updating,
                       const bool *visited, int *cost, int n) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n && mask[tid]) {                          // data-dependent branch
        mask[tid] = false;
        for (int i = nodes[tid].starting; i < nodes[tid].starting + nodes[tid].no_of_edges; i++) {
            int id = edges[i];                           // indirect: i comes from nodes[]
            if (!visited[id]) {                          // indirect, and a data-dependent branch
                cost[id] = cost[tid] + 1;                // indirect write
                updating[id] = true;
            }
        }
    }
}
```

Figure 1: The frontier expansion of a breadth-first search (`rodinia_bfs`, one of our test programs). The bytes the kernel reads and writes depend on the graph and on what earlier launches wrote.

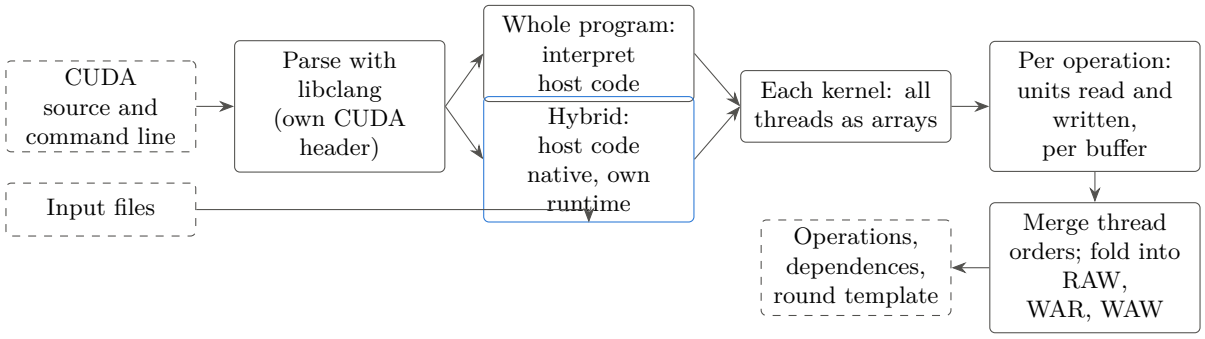


Figure 2: The steps of the two interpreters. Nothing runs on the GPU. The input reaches the whole-program interpreter through the interpreted host code, and the hybrid interpreter through the native host code.

3 The interpreter

3.1 Overview

Both interpreters take the program's CUDA source files and its command line, run the program on the CPU from `main()`, and need no GPU. They differ in how they run the host code. Figure 2 shows the steps. First, libclang [15] parses the source into a syntax tree. CUDA's own headers are replaced by a small header of our own that declares the runtime functions (`cudaMalloc`, `cudaMemcpy` and so on). Second, the program runs. The whole-program interpreter executes the host code's syntax tree as MemPrint's C++ interpreter does [16] (Section 3.2); the hybrid interpreter compiles the host code and runs it natively (Section 3.3). In both, each call to a CUDA runtime function or kernel launch becomes an operation, and every kernel is interpreted from its syntax tree (Section 3.4). Third, each operation records the units it read and wrote in each buffer. Fourth, the operations are folded, in order, into the dependences between them. The program runs several times, under different thread orders, and the runs are merged (Section 3.8). The output is a list of operations, a list of dependences with their kinds, buffers, unit counts and certainty, a template of the dependences of rounds whose number the thread order decides, and optionally the address ranges each operation touched.

3.2 Running the host program

The host part of a CUDA program is ordinary C++, and the interpreter runs it the way MemPrint's C++ interpreter does [16]. Every allocation, global variable and the stack is a memory *object*. A pointer is a pair of an object and a byte offset into it. The interpreter keeps the integer value stored in every 4-byte unit of every object, and remembers which units hold pointers and to which object they point. As a result, when the program reads a graph from a file into an array and later indexes another array with an element of it, the interpreter knows the index. Floating-point values in memory are not kept, and a value the interpreter does not know is marked unknown rather than guessed.

Device memory fits into the same scheme. Each `cudaMalloc` creates a new object of kind *device*, and each `__device__` or `__constant__` variable is an object as well. A `cudaMemcpy` copies the stored values, and the pointer information, from one object to the other, so an index array built on the host and copied to the GPU keeps its contents, and a table of device pointers built on the host still points to the right buffers after the copy. A `cudaMemset` stores its value in the filled units.

A loop on the host that issues GPU work runs one iteration at a time, in program order. The interpreter otherwise runs short counted loops as one step over all iterations (Section 3.4), and we changed it so that an allocation, copy or fill inside such a loop is carried out once per iteration, because each iteration creates a different buffer or a different operation.

3.3 Running the host program natively: the hybrid interpreter

The whole-program interpreter must understand every line of the host code. Real host code reads its input with C++ streams and strings, uses library containers, or is written in old C dialects, and where the interpreter does not model what the code does it computes a wrong execution: in our benchmarks, a graph read through `std::ifstream` came out empty. The hybrid interpreter avoids interpreting the host code at all.

The program is compiled by `nvcc` as a shared library, with its `main()`, against a stand-in for the CUDA runtime instead of CUDA's own. In the stand-in runtime, device memory is ordinary host memory: `cudaMalloc` allocates zeroed host memory, a copy or fill is a `memcpy` or `memset`, and `__device__` and `__constant__` variables live in the host copy that `nvcc` creates for each of them. The interpreter loads the library into its own process and calls the program's `main()`, so the host code runs at native speed with every library it uses. Every allocation, copy, fill and launch calls back into the interpreter. At a launch, the interpreter binds the kernel's arguments from the launch's argument array (a pointer argument becomes the buffer it points into, a structure passed by value is copied field by field) and runs the kernel as in Section 3.4. The kernel's loads read the program's real bytes, with the signedness of their C type, and its stores write them, so the host code continues from the kernel's results. Every value is concrete, so the hybrid interpreter knows values that the whole-program interpreter does not, such as those of `drand48`.

Two pieces of information that the whole-program interpreter takes from the host code are recovered differently. Whether an operation was issued under data control (Section 3.5) is decided from the source for each call site; the stand-in runtime records the return address of every copy and fill, and `addr2line` maps it to its line. The host side of a copy, which the whole-program interpreter tracks as an object, is tracked as a region of host memory named in order of first use; locals on the stack are left out, as the whole-program interpreter keeps scalar locals out of memory. Because the program's native state cannot be reset, each thread order runs in a process of its own, and the processes' results are merged.

3.4 Running a kernel: all threads at once

A kernel launch runs the kernel's body once, for all of its threads together. Every variable of the kernel holds an array with one value per thread, and every statement is carried out for all threads before the next statement starts. MemPrint's interpreter uses the same representation to run all iterations of a loop at once, and for a kernel the threads take the place of the iterations. The thread's `threadIdx.x` is an array $0, 1, 2, \ldots$; an expression such as `blockIdx.x * blockDim.x + threadIdx.x` is computed for all threads with one array operation; and `edges[i]`, where `i` differs per thread, loads one element per thread from the stored values of the object `edges` points to.

A branch splits the threads. The interpreter evaluates the condition for every thread, runs the branch only for the threads where it holds, and merges the variables of the two groups afterwards. A loop whose trip count differs per thread runs until no thread continues, and a thread that returns from the kernel stops taking part. In this way every thread follows its own control path, decided by the values it loaded.

We call this order of execution *lockstep*: statement by statement, all threads at a time. Lockstep is one order in which the GPU could legally run the threads of a kernel that has no data races, because the GPU guarantees nothing about the relative progress of threads between barriers. A barrier, `__syncthreads()`, has no work to do in lockstep, since every thread has already reached it when the statement before it finishes. Each block has its own copy of the kernel's `__shared__` arrays. Atomic operations to the same address from different threads take effect one after another in thread order, and each thread receives the value that was there before its own update, as on the GPU. The interpreter runs the blocks of a large grid in groups of at most 2^{18} threads to bound its memory use.

3.5 Recording what each operation touches

Every operation records, per object and per byte, whether it read the byte and whether it wrote it. A kernel records every load and store that its threads carry out, a copy records the bytes it reads at the source and writes at the destination, and a fill records the bytes it writes. A read of a byte that the operation has already written for certain is not recorded as a read, because it reads the operation's own value, not an earlier operation's: in `tmp[i] = 0; tmp[i] += a[i][j] * x[j];` the kernel does not depend on whoever wrote `tmp` before it. The interpreter also records the host code between two operations as an operation of its own, which a tool can ignore; the evaluation leaves it out.

A record has two further marks. The first says whether the access is only possible, which is the case when the interpreter could not compute the access exactly. An address it could not compute covers the whole object it points into, and an access under a branch whose condition is unknown is recorded but marked as possible. An access through a pointer whose object is unknown is recorded as a possible access to every device buffer, and so is an access past the end (or before the start) of a device buffer. On the GPU such an access reaches whatever memory lies there, often the next buffer, which the interpreter cannot know; it reports where the access happens, because it is usually a bug in the program. The second mark says whether the access is data-dependent (Section 2). The interpreter decides this from the source code before the program runs, by marking every value that is loaded from memory, or computed from such a value, or assigned under a condition that is, and repeating until nothing changes. An access whose address or whose enclosing conditions use a marked value is data-dependent, and so is every access of an operation that the host issued from a loop whose condition depends on device data, such as a loop that runs until the GPU reports that nothing changed.

```
init<<<b, t>>>(a, 1.0f, n);
init<<<b, t>>>(b, 2.0f, n);
add<<<b, t>>>(a, b, c, n);     // c = a + b
scale<<<4, 64>>>(c, n);        // c *= 2
cudaMemcpy(h, c, n * 4,
           cudaMemcpyDeviceToHost);
```

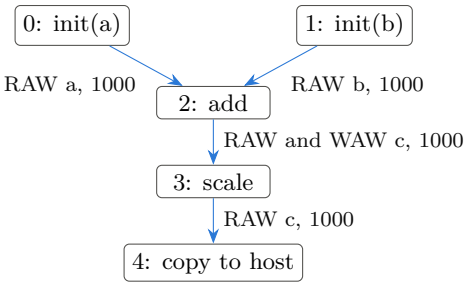


Figure 3: The operations of `chain` with $n = 1000$ and the dependences the interpreter reports, with the number of 4-byte units each passes through. The two `init` launches are independent and could run at the same time. All dependences are certain, and all are needed.

```
__global__ void distribute(const int *keys, int *const *bucket, int *count, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    int k = keys[i] % 4;
    bucket[k][atomicAdd(&count[k], 1)] = i;   // the buffer itself is chosen by keys[i]
}
```

Figure 4: A kernel that writes each element into one of four buffers, through a table of pointers in device memory (`ptrsel`). The four consumer kernels each read one buffer.

3.6 From touched bytes to dependences

The operations are folded into dependences in the order they were issued. For every unit of every object the interpreter keeps the last operation that certainly wrote it, the operations that possibly wrote it since then, and the operations that read it since then. When the next operation arrives, every unit it reads produces a RAW dependence on the last writer and on each possible writer since, every unit it writes produces a WAW dependence on them and a WAR dependence on each reader, and then the operation's own writes replace the history of those units. A dependence is certain if both of its accesses are certain and no operation possibly wrote the unit in between, and possible otherwise. The counts of units are summed per dependence. Figure 3 shows the result for a short program.

3.7 Threads that use different buffers

Some kernels choose a buffer, not only an element, from data. Figure 4 shows a kernel that distributes elements into one of four buckets, each a separate buffer, through a table of device pointers that the host copied to the GPU. In lockstep, the statement `bucket[k][...] = i` loads one pointer per thread, and the pointers of different threads point to different buffers. The interpreter's pointer is a pair of one object and an offset, so it cannot represent "buffer 0 for these threads and buffer 3 for those". Before the change described here, such a pointer became unknown, every write through it became a possible write to every device buffer, and every dependence from the kernel to the four consumers was only possible.

We resolve this by splitting the threads. When a kernel loads pointers from memory and the threads' pointers name different objects, the interpreter groups the threads by the object their pointer names, and then runs the statement, and the rest of the enclosing block of code, once for each group, with only that group's threads active. Inside a group the pointer names one object, so every access through it is exact. After the last group, the variables of all groups are merged as after

```
__global__ void hook(const int *src, const int *dst, int *label, int *changed, int m) {
    int e = blockIdx.x * blockDim.x + threadIdx.x;
    if (e >= m) return;
    int a = label[src[e]], b = label[dst[e]];   // may see another thread's update, or not
    if (a < b && atomicMin(&label[dst[e]], a) > a) *changed = 1;
    if (b < a && atomicMin(&label[src[e]], b) > b) *changed = 1;
}
```

Figure 5: One round of connected components by label propagation (`cc`). The host launches it until `changed` stays zero.

a branch. A pointer variable that ends up naming different objects in different groups becomes unknown at the merge, which is safe.

Running a statement a second time is correct only if its first, interrupted execution changed nothing. The interpreter therefore counts stores and atomic operations, notes the count when each statement starts, and splits only if the count is unchanged when the pointer is loaded. If a store already happened, the pointer stays unknown as before. For `bucket[k][atomicAdd(...)]` the split happens, because C++17 evaluates the array expression `bucket[k]` before the index, so the pointer is loaded before the atomic operation [13]. A pointer loaded after an atomic operation in the same statement cannot be split. The interpreter also limits the split to at most 64 different objects.

3.8 Kernels whose result depends on thread order

Lockstep is a legal order of execution, but it is not the only one, and for some kernels the bytes touched depend on the order. The connected-components kernel in Figure 5 lowers labels in place. Thread e reads the labels of the two ends of edge e and writes the smaller into the larger with `atomicMin`. If another thread lowered one of those labels earlier in the same launch, thread e reads the new value, otherwise the old one, so which threads write, and which units, depends on the order. The host repeats the kernel until a launch changes nothing, so the number of launches depends on the order too. On the GPU, 60 runs of this program took 21 to 24 rounds, and each of 10 traced runs touched different bytes. In lockstep, every thread reads its labels before any thread of the launch writes one, so no update reaches another thread within the same round, and the program takes 24 rounds. With lockstep alone, the interpreter reported all of its dependences as certain, and 1% to 21% of them did not occur in a given GPU run (Section 5.5).

Several legal orders. The interpreter runs the whole program under four thread orders. The first is lockstep. In the other three, each block runs to completion before the next starts, in the order of block numbers, in reverse order, or in a random order, and within a block the warps of 32 threads run one after another in the same order, unless the kernel uses `__syncthreads` or shared memory, which require the warps of a block to run together. A later block or warp then sees what earlier ones wrote. Only some kernels need these orders. A launch in which no unit is both read and written (by any thread, an atomic counting as both) computes the same result in any order, so the lockstep run notes which kernels had a launch that read and wrote one unit, and the other orders reorder only those and run the rest in lockstep.

Merging the runs. We match the operations of the runs by their common beginning and their common end, because a loop that runs a different number of rounds adds or removes operations in

the middle of the sequence. A dependence stays certain only if every run has it between operations that every run has, every run reports it as certain, and it does not lead from before the point where the runs part to after the rounds that differ, since those rounds decide which operation last wrote each unit. All other dependences of the first run, and those that only other runs have, become possible and are marked as depending on the schedule.

What other orders could do. Four orders are a sample. A GPU run can interleave threads differently and produce a dependence none of the four produced, and a tool that relies on the result must not miss it. The interpreter therefore over-approximates where the orders disagree. Each run keeps, per operation and buffer, the units read and written. For an operation that every run issues but that touched different units in different runs, the interpreter takes the union over the runs, and adds a possible dependence between it and every other operation whose union conflicts with it (one writes a unit the other reads or writes). Where the runs issue different numbers of operations, the interpreter finds the shortest repeating sequence of operations that covers the part where they differ, such as *fill the flag, launch the kernel, copy the flag back*, and writes a *round template*. For each position in a round it takes the union of the units touched over every round of every run, and it records every conflict as a possible dependence: between two positions in one round, from a round to any later round, from an operation before the loop to every round, from every round to an operation after the loop, and between the operations before and after the loop. A tool instantiates the template for the number of rounds the program actually runs; the scorer does so for the traced run. This over-approximation removes missed dependences at the cost of reporting dependences that a given run does not have.

3.9 What the interpreters cannot know

Both interpreters need the program's input, because the bytes an irregular kernel touches are a function of it. In a kernel they do not model floating-point values stored in memory or a pointer loaded in a statement after a store, and accesses that depend on such values are recorded as possible. The whole-program interpreter also does not model every library function the host code calls (for example `drand48`, C++ file streams and strings); the `maybe` test program checks the first case. When the host code calls such functions, the values the program computes from them are unknown or wrong, and the operations it issues may be too. The interpreter counts these calls and its parse errors and warns that the result is unreliable, but it cannot recover the operations it did not see; the hybrid interpreter, which runs the host code natively, is the remedy. A kernel whose result depends on thread order is handled by sampling orders and over-approximating what other orders could do (Section 3.8).

4 Measuring the truth on a GPU

To measure the interpreter's accuracy, we need the bytes each operation actually touches on a GPU. We built an NVBit [20] tool for this. NVBit is a framework that inserts code before chosen machine instructions of a kernel when the kernel is loaded. Our tool inserts, before every load, store and atomic operation on global memory, a call that sends the addresses of the 32 threads of the warp, the access size and whether it reads or writes, together with the number of the current launch, to a buffer that a CPU thread empties. On the host side the tool intercepts the CUDA driver's functions for allocation, copies, fills and kernel launches, so it knows every buffer's base address and size and every operation's position. It writes, per operation and buffer, the ranges of units read and written, in the same format the interpreter can write.

The tool was written against NVBit 1.7 on a machine without a GPU, and it needed three changes to work with NVBit 1.8 and CUDA 13.4. First, the GPU side of the buffer must be flushed explicitly after each launch and at the end, with a small kernel loaded as a separate module, otherwise the last partly filled buffer, which for a small program holds all records, is lost. Second, the launch number must be set before the instrumented code is enabled for the launch, otherwise the records carry a wrong number. Third, the CUDA 13 runtime may launch kernels through `cuLaunchKernelEx`, which the tool must intercept as well. In addition, the interpreter's stand-in CUDA header had to declare the launch function that clang uses when it finds a recent CUDA installation on the machine.

Comparing two runs. The traced run's bytes are folded into dependences by the same code as the interpreter's, so the two sides differ only in which bytes each operation touched. Buffers are matched by the order of their allocation, and operations by their order. When the two runs issue different numbers of operations, as `cc` does, operations are matched by common beginning and common end as in Section 3.8, the interpreter's round template is instantiated for the traced run's rounds and added as possible dependences, and a traced dependence through an operation that remains unmatched counts as missed. A dependence is identified by its source and destination operation, its kind and its buffer. Comparing the interpreter's dependences with the traced run's gives five counts: certain dependences the run has (true positives) and does not have (false positives), possible dependences the run has and does not have, and the run's dependences the interpreter does not report at all (false negatives). From them we compute three scores. *Certain precision* is the share of the interpreter's certain dependences that the traced run has. *Recall* is the share of the traced run's dependences that the interpreter reports, certain or possible. *Certain recall* is the share it reports as certain. *Units error* is, over the dependences both have, the summed difference in unit counts divided by the traced units. Dependences through memory that is not a `cudaMalloc` buffer (`__device__` variables, host memory) are not scored, since the tool does not see them as buffers. We also compute the scores over the needed dependences of each side.

5 Evaluation

5.1 Setup

We ran the GPU programs on an NVIDIA A30 (compute capability 8.0) with driver 610.57, CUDA 13.4 and NVBit 1.8, and the interpreters on the same machine's CPU, with Python 3.10, NumPy and libclang 18.1. The programs were compiled with `nvcc -O2`. Table 1 lists the 14 test programs. They are small, between 22 and 103 lines of code, and each exercises one pattern: regular elementwise kernels, shared memory, scattered writes through host-built or file-provided indices, atomics, values the interpreter does not model, three versions of breadth-first search (one kernel per level, Rodinia's two-kernel form with a graph file, and a frontier queue filled with `atomicAdd`), buffers chosen through pointer tables, `__device__` and `__constant__` variables written with symbol copies, and connected components. The graphs have 4,096 vertices with 1 to 5 random neighbours each. We wrote the programs together with the interpreter, so they test the patterns it was built for; Section 7 discusses what this does not show.

5.2 Accuracy of the dependences

Table 2 compares both interpreters, run under all four thread orders, with one traced run of each test program. *Neither interpreter misses a dependence, and every dependence either reports as certain*

Program	What it exercises	Lines	Kernels	Operations	Dependences
`chain`	elementwise kernels in a chain	32	3	5	5
`reduce`	a reduction through shared memory	39	2	5	4
`stencil`	a stencil repeated over two buffers	22	1	7	16
`scatter`	writes at indices built on the host	35	3	6	5
`fileidx`	indices and weights read from files	41	2	5	6
`histogram`	atomic updates of bins chosen by the data	45	2	10	14
`maybe`	a branch on values the interpreter does not model	30	2	4	5
`bfs`	breadth-first search, one kernel per level	50	1	49	359
`rodinia_bfs`	breadth-first search in the style of Rodinia	103	2	59	609
`wl_bfs`	breadth-first search with a frontier queue	60	1	56	345
`ptrsel`	a buffer chosen by the data through a pointer table	35	2	10	21
`ptrsel2`	pointer fields in structs, pointers in variables	88	4	16	29
`symbols`	device and constant variables, symbol copies	46	3	8	6
`cc`	connected components, labels lowered in place	49	2	76	610

Table 1: The test programs. Lines counts non-empty, non-comment lines. Operations is the number of kernel launches, copies and fills the program issues, and dependences the number of dependences on the GPU.

occurs on the GPU: recall and certain precision are 1.00 for all 14 programs, over 1,595 certain dependences. In 11 of the 14 programs the whole-program interpreter reports every dependence as certain and the unit counts match exactly; the hybrid interpreter does so in 12, and otherwise gives the same result.

The exceptions are the programs built to contain what the interpreters cannot resolve exactly. In `maybe`, a branch on `drand48` values makes three dependences possible for the whole-program interpreter, which does not model `drand48`, and one of them did not occur on the GPU. The hybrid interpreter runs `drand48` natively, knows the values and reports all five dependences as certain. In `ptrsel2`, one kernel loads its pointer after an atomic operation in the same statement, so its threads cannot be split (Section 3.7), and its accesses are possible accesses to every device buffer; the other three kernels of the program are exact. In `cc`, the dependences of the rounds whose touched bytes depend on thread order are possible, and the over-approximation of Section 3.8 adds every dependence another order could produce: 3,047 possible dependences against the traced run's 610 (Section 5.5).

5.3 Dependences through data-dependent addresses and branches

Table 3 restricts the comparison to dependences between two kernels that pass through bytes touched by an indirect access or under a data-dependent branch, the dependences a compiler cannot determine. There are 1,427 of them across the programs. The interpreter reports 999 as certain, and all of these occur on the GPU. In the three breadth-first searches, every one of the 868 such dependences is certain and correct, including those through the frontier queue of `wl_bfs`, whose slots threads claim with `atomicAdd` in an order that differs between runs. The order changes which vertex lands in which slot, but not the set of slots written or the set of vertices whose distances the next level reads, so the dependences between launches do not change. In `ptrsel`, the dependence of each consumer on the bucket it reads is certain, with the exact number of elements, and the consumer of the bucket that stays empty has no dependence on the producer. The remaining dependences of this kind are the possible ones of `maybe`, `ptrsel2` and `cc` described above.

Program	Whole program					Hybrid					Ranges (whole program)		
	Cert.	Poss.	Cert. prec.	Recall	Cert. recall	Cert.	Poss.	Cert. prec.	Recall	Cert. recall	Exact sets	Unit prec.	Unit recall
`chain`	5	0	1.00	1.00	1.00	5	0	1.00	1.00	1.00	8/8	1.00	1.00
`reduce`	4	0	1.00	1.00	1.00	4	0	1.00	1.00	1.00	7/7	1.00	1.00
`stencil`	16	0	1.00	1.00	1.00	16	0	1.00	1.00	1.00	11/11	1.00	1.00
`scatter`	5	0	1.00	1.00	1.00	5	0	1.00	1.00	1.00	9/9	1.00	1.00
`fileidx`	6	0	1.00	1.00	1.00	6	0	1.00	1.00	1.00	8/8	1.00	1.00
`histogram`	14	0	1.00	1.00	1.00	14	0	1.00	1.00	1.00	17/17	1.00	1.00
`maybe`	2	3	1.00	1.00	0.40	5	0	1.00	1.00	1.00	7/8	0.94	1.00
`bfs`	359	0	1.00	1.00	1.00	359	0	1.00	1.00	1.00	108/108	1.00	1.00
`rodinia_bfs`	609	0	1.00	1.00	1.00	609	0	1.00	1.00	1.00	195/195	1.00	1.00
`wl_bfs`	345	0	1.00	1.00	1.00	345	0	1.00	1.00	1.00	172/172	1.00	1.00
`ptrsel`	21	0	1.00	1.00	1.00	21	0	1.00	1.00	1.00	25/25	1.00	1.00
`ptrsel2`	21	36	1.00	1.00	0.72	21	36	1.00	1.00	0.72	31/49	0.57	1.00
`symbols`	6	0	1.00	1.00	1.00	6	0	1.00	1.00	1.00	8/8	1.00	1.00
`cc`	182	3047	1.00	1.00	0.30	182	3047	1.00	1.00	0.30	134/171	0.89	0.97

Table 2: Both interpreters against one traced run per program. Cert. and Poss. count the interpreter's certain and possible dependences. Certain precision, recall and certain recall are defined in Section 4. The address-range columns compare, for every operation, buffer, and read or write, the set of units the whole-program interpreter touched with the GPU's: Exact sets counts how many of these sets are identical, and unit precision and recall are the shares of the interpreter's units the GPU touched and of the GPU's units the interpreter reported.

Program	Between kernels	Through data-dependent bytes	Reported certain	Certain and in the trace	In the trace (any)
`scatter`	1	1	1	1	1
`fileidx`	1	1	1	1	1
`histogram`	1	1	1	1	1
`maybe`	2	1	0	0	1
`bfs`	233	233	233	233	233
`rodinia_bfs`	454	454	454	454	454
`wl_bfs`	181	181	181	181	181
`ptrsel`	6	6	6	6	6
`ptrsel2`	22	11	2	2	6
`cc`	1152	538	120	120	445
Total	2068	1427	999	999	1329

Table 3: Dependences between two kernels (copies and fills excluded), and among them those through bytes reached by an indirect access or under a data-dependent branch. The last two columns count how many of the latter the traced run has. Programs with none are omitted.

	Exact units		One interval		Whole allocation	
Program	Pairs	Needed	Pairs	Needed	Pairs	Needed
`chain`	5	4	5	4	5	4
`reduce`	4	4	4	4	4	4
`stencil`	17	6	17	6	17	6
`scatter`	6	5	7	5	7	5
`fileidx`	5	4	5	4	5	4
`histogram`	13	9	13	9	13	9
`maybe`	4	3	4	3	4	3
`bfs`	916	48	916	48	916	48
`rodinia_bfs`	1009	69	1009	69	1009	69
`wl_bfs`	1185	55	1190	55	1191	55
`ptrsel`	17	13	18	13	21	9
`ptrsel2`	26	18	26	18	27	15
`symbols`	6	4	6	4	6	4
`cc`	1958	69	1958	69	1958	69

Table 4: Pairs of operations that conflict in the traced run (both touch a unit and one writes it), and the needed pairs among them, when each operation's touched units are taken exactly, as one interval per buffer, or as the whole buffer.

Address ranges. The right part of Table 2 compares the units each operation touched, not only the dependences. For 12 programs every set of units is identical to the GPU's. The interpreter's sets are larger only where it reports possible accesses, in `maybe` (512 units possibly written, 254 written) and `ptrsel2` (the kernel whose pointer could not be split covers every buffer), and they differ in `cc`, whose rounds touch different units in each run.

How much the exact ranges add. In our programs, most kernels touch all or most of every buffer they use, so tracking whole buffers instead of exact units gives the same dependences for 11 of the 14 programs (Table 4). Exact ranges matter in `ptrsel` and `ptrsel2`, where several kernels touch disjoint parts of shared buffers, and they remove one false pair in `scatter` and `wl_bfs`. The test programs therefore show that the interpreter finds the exact bytes, but not that exact bytes are needed; programs whose kernels partition buffers, such as blocked matrix factorisations, would show that.

5.4 Unmodified benchmarks

The test programs exercise one pattern each and were written together with the interpreters. To see how the interpreters fare on code we did not write, we ran them on 38 unmodified programs from three suites: 8 from Rodinia [7] (`bfs` with a graph of 4,096 vertices, `gaussian`, `nw`, `pathfinder`, `hotspot` with generated temperature and power files, `lud`, `backprop` and `srad`), the 10 GPU programs of GraphBIG [17] on its small social-network graph (1,000 vertices, 29,790 edges), and 20 programs of PolyBench-GPU [11]. Inputs were chosen small enough for the tracer. The sources were not changed, except where CUDA 13 no longer has an API they use: `cudaThreadSynchronize` is defined as `cudaDeviceSynchronize`, two `printf` calls of `gaussian` that print removed device properties are commented out, and PolyBench's problem sizes are set in its headers. Rodinia's `kmeans` is left out because it uses texture references, which CUDA 13 removed. Each program was traced once with NVBit, at 2.0 to 219 times its run time, and each interpreter ran it under the four thread orders, 12 interpreter runs at a time, with a limit of two hours per run.

Interpreter	Suite	Programs	GPU deps	Certain		Possible		Missed (FN)
				On the GPU	Not on the GPU	On the GPU	Not on the GPU	
Hybrid	Rodinia	8	1,489	1,488	0	1	4	0
	GraphBIG	9	8,829	8,690	0	139	1,439	0
	PolyBench-GPU	20	5,674	5,674	0	0	0	0
	Total	37	15,992	15,852	0	140	1,443	0
Whole program	Rodinia	8	1,489	1,436	0	16	20	37
	GraphBIG	9	8,829	1	0	86	391	8,742
	PolyBench-GPU	20	5,674	5,674	0	0	0	0
	Total	37	15,992	7,111	0	102	411	8,779

Table 5: The interpreters against one traced run of each benchmark, summed per suite: the GPU's dependences, the interpreter's certain and possible dependences split by whether the run has them, and the run's dependences the interpreter does not report (false negatives). GraphBIG's `betweenness` timed out in both interpreters and is not included.

Table 5 sums the results per suite and Table 6 lists every program. *The hybrid interpreter misses none of the GPU's dependences and reports no certain dependence that the GPU does not have*, on the 37 programs it completes. Of the 15,992 dependences on the GPU it reports 15,852 as certain and 140 as possible. On 33 programs its result is exact: every dependence certain, every unit count equal to the GPU's, nothing reported that did not occur. The 1,443 possible dependences that the traced run did not have come from four programs. In `srad`, two kernels read one row past the end of their input arrays (`srad_kernel.cu` lines 45, 46, 55 and 200), which on the GPU reaches the next buffer; the interpreter reports the out-of-bounds reads and makes them possible accesses to every buffer. In GraphBIG's `bfs-topo-atomic`, `bfs-data-thread-centric` and `connected-comp`, threads lower values in place with atomics, so the number of rounds or the units touched depend on thread order, and the over-approximation of Section 3.8 adds every dependence another order could produce.

The whole-program interpreter is exact on the 26 programs whose host code it can run: six of the Rodinia programs and all of PolyBench. It misses 8,779 dependences, all in the 11 programs whose host code it cannot run. GraphBIG reads its graph through the openG library, with `std::ifstream`, `getline` and `std::string`, which the interpreter does not model, so the graph comes out empty. Rodinia's `backprop` uses K&R-style function definitions, which do not parse as C++. Rodinia's `lud` reads its options with `getopt_long`, which the interpreter does not model, so it runs a different matrix size. The interpreter warns on all 11, and on three programs where its result is nevertheless exact. It reports no certain dependence the GPU does not have, here as on the test programs.

GraphBIG's `betweenness`, whose traced run has 165,863 dependences, ten times more than any other program, did not finish in two hours in either interpreter. The other programs took 9 to 941 seconds in the hybrid interpreter (median 27), with 12 runs sharing the machine.

5.5 Kernels whose result depends on thread order

Figure 6 shows how many rounds `cc` takes. On the GPU, 50 plain runs and 10 runs under NVBit took between 21 and 24 rounds. The interpreter's orders took 11 to 24 rounds. Lockstep takes the most rounds, because no label update reaches another thread within a round. Running blocks and warps one after another lets labels flow from earlier ones to later ones within a round, which takes fewer rounds. The GPU, which runs many blocks at once but not all of them, lies between the two, inside the interpreter's range.

Suite	Program	Ops	GPU deps	Hybrid Cert.	Hybrid Poss. yes	Hybrid Poss. no	Hybrid FN	Hybrid s	Whole program Cert.	Whole program Poss. yes	Whole program Poss. no	Whole program FN	Whole program s
Rodinia	`bfs`	59	609	609	0	0	0	101	609	0	0	0	218
Rodinia	`gaussian`	132	696	696	0	0	0	162	696	0	0	0	147
Rodinia	`nw`	18	81	81	0	0	0	33	81	0	0	0	24
Rodinia	`pathfinder`	7	15	15	0	0	0	12	15	0	0	0	5
Rodinia	`hotspot`	4	3	3	0	0	0	12	3	0	0	0	9
Rodinia	`lud`	12	37	37	0	0	0	15	0	15	16	22	5
Rodinia	`backprop`	10	15	15	0	0	0	44	0	0	0	15	3
Rodinia	`srad`	8	33	32	1	4	0	16	32	1	4	0	9
GraphBIG	`bfs-topo-thread-centric`	20	64	64	0	0	0	64	0	6	7	58	9
GraphBIG	`bfs-topo-atomic`	17	44	26	18	120	0	23	0	6	7	38	9
GraphBIG	`bfs-data-thread-centric`	24	80	80	0	66	0	124	1	11	10	68	31
GraphBIG	`sssp-topo-thread-centric`	27	153	153	0	0	0	23	0	19	47	134	8
GraphBIG	`connected-comp`	58	267	146	121	1,253	0	201	0	17	196	250	9
GraphBIG	`kcore`	15	32	32	0	0	0	20	0	11	68	21	8
GraphBIG	`gc-topo-thread-centric`	263	8,167	8,167	0	0	0	845	0	4	8	8,163	7
GraphBIG	`degree-centr`	5	6	6	0	0	0	17	0	2	1	4	7
GraphBIG	`triangle-count`	9	16	16	0	0	0	941	0	10	47	6	9
GraphBIG	`betweenness`	–	–	timeout					timeout				
PolyBench	`2dconv`	3	2	2	0	0	0	10	2	0	0	0	17
PolyBench	`2mm`	8	8	8	0	0	0	30	8	0	0	0	42
PolyBench	`3dconv`	33	91	91	0	0	0	13	91	0	0	0	23
PolyBench	`3mm`	11	10	10	0	0	0	12	10	0	0	0	19
PolyBench	`atax`	7	7	7	0	0	0	20	7	0	0	0	24
PolyBench	`bicg`	9	8	8	0	0	0	11	8	0	0	0	19
PolyBench	`corr`	10	7	7	0	0	0	18	7	0	0	0	25
PolyBench	`covar`	7	4	4	0	0	0	11	4	0	0	0	17
PolyBench	`fdtd-2d`	35	178	178	0	0	0	915	178	0	0	0	931
PolyBench	`gemm`	5	5	5	0	0	0	27	5	0	0	0	31
PolyBench	`gesummv`	7	8	8	0	0	0	11	8	0	0	0	20
PolyBench	`gramschm`	386	893	893	0	0	0	46	893	0	0	0	54
PolyBench	`jacobi1d`	24	72	72	0	0	0	9	72	0	0	0	17
PolyBench	`jacobi2d`	44	142	142	0	0	0	205	142	0	0	0	259
PolyBench	`lu`	258	761	761	0	0	0	314	761	0	0	0	317
PolyBench	`mvt`	9	10	10	0	0	0	24	10	0	0	0	26
PolyBench	`syr2k`	5	5	5	0	0	0	37	5	0	0	0	41
PolyBench	`syrk`	4	4	4	0	0	0	27	4	0	0	0	39
PolyBench	`gemver`	13	16	16	0	0	0	165	16	0	0	0	145
PolyBench	`adi`	262	3,443	3,443	0	0	0	51	3,443	0	0	0	50

Table 6: Every benchmark. Ops is the number of operations in the traced run, GPU deps its dependences. Per interpreter: certain dependences (all of which the traced run has), possible dependences the run has (Poss. yes) and does not have (Poss. no), the run's dependences not reported (FN), and the interpreter's run time in seconds under four thread orders.

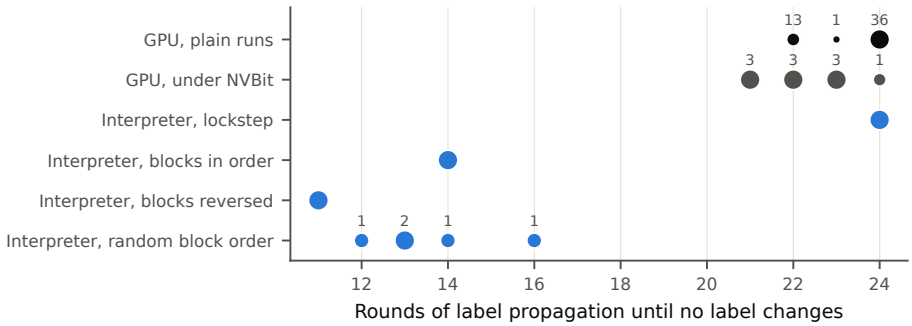


Figure 6: Rounds of `cc` until no label changes, on the GPU and in the interpreter. Each dot is a round count, its size and the number above it give how many runs produced it. The random order was run with five seeds.

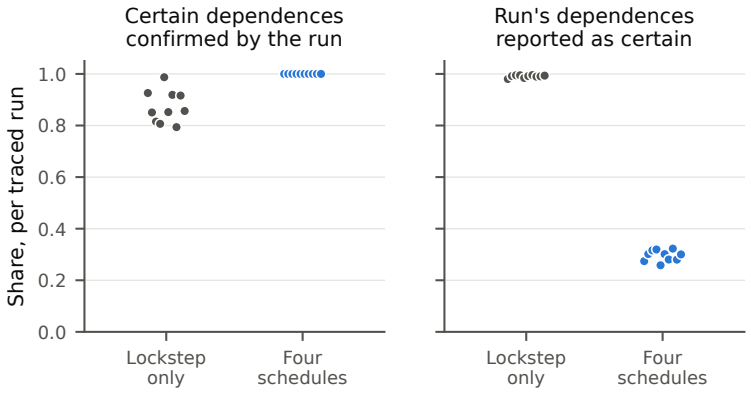


Figure 7: `cc` against each traced run: the share of the interpreter's certain dependences that the run has (left), and the share of the run's dependences that the interpreter reports as certain (right). Lockstep alone reports false certain dependences, and the four orders together do not.

Figure 7 scores the interpreter against each of the 10 traced runs. With lockstep alone, the interpreter reports all of its dependences as certain, and only 0.79 to 0.99 of them occur in a given run, so the rest are false. With the four orders merged and the round template added, the certain precision is 1.00 and the recall is 1.00 against every run: no run has a dependence the interpreter does not report. The interpreter reports 0.26 to 0.32 of each run's dependences as certain, mostly those of the setup and of the first round, which every order shares. The price of the recall is in the possible dependences: the round template allows every round to depend on every earlier round through every unit any round touched, which gives 3,047 possible dependences against a traced run's 610. Without the over-approximation, merging the four orders alone left 0.5% to 2% of each run's dependences unreported, for example a label written in one round and next overwritten three rounds later.

5.6 What each mechanism contributes

Table 7 turns each of the two mechanisms off in turn. Without splitting threads by buffer, every dependence through the buckets of `ptrsel` becomes possible, so only 10 of the 21 traced dependences are reported as certain and only half of the needed ones; in `ptrsel2`, certain recall falls from 0.72 to 0.14 and certain recall of the needed dependences from 0.81 to 0.10. Recall stays at 1.00,

Program	Interpreter	Certain	Possible	Certain precision	Recall	Certain recall	Needed cert. recall	Units error
`ptrsel`	Full interpreter	21	0	1.00	1.00	1.00	1.00	0.00
`ptrsel`	Without thread splitting	10	22	1.00	1.00	0.48	0.50	0.00
`ptrsel2`	Full interpreter	21	36	1.00	1.00	0.72	0.81	0.07
`ptrsel2`	Without thread splitting	4	105	1.00	1.00	0.14	0.10	0.47
`cc`	Full interpreter	182	3047	1.00	1.00	0.30	0.49	0.33
`cc`	Lockstep schedule only	703	0	0.85	0.98	0.98	0.96	0.10

Table 7: The interpreter with one mechanism turned off, against the same traced runs as Table 2.

because the unknown pointer makes the accesses possible accesses to every buffer, which includes the right one. Without the extra thread orders, `cc` reports false certain dependences, as Section 5.5 showed, and misses 2% of the run's dependences.

5.7 Overhead

The interpreters' cost is their run time on the CPU. Figure 8 and Table 8 compare it with the program's own run on the GPU and with the run under the NVBit tool, for six programs at three input sizes each. The GPU runs take 0.29 to 0.44 seconds, and almost all of that time is the start-up of the CUDA runtime, because at these sizes the kernels themselves take milliseconds. The NVBit tool takes 2.0 to 12.6 times as long as the GPU run. Against the kernels alone, both interpreters are several orders of magnitude slower, as expected from simulating every thread in Python on one CPU core.

In lockstep. *The hybrid interpreter in lockstep takes 1.1 to 7.0 seconds, 3.0 to 16 times the GPU run (median 4.5).* Most of this is a fixed cost of about one second for starting Python and parsing the kernels; the rest is running the kernels. The whole-program interpreter takes 0.7 to 660 times the GPU run, and the difference is host code. The kernels of `chain`, with 3,000,448 GPU threads, take 4.7 seconds in both, and those of `rodinia_bfs`, with 2,228,224 threads, take less than one second, so `rodinia_bfs` as a whole takes 3.1 seconds, against 2.3 seconds under NVBit. But `wl_bfs` takes 229 seconds in the whole-program interpreter, of which 0.26 in kernels, and `bfs` 136 seconds, of which 0.9 in kernels: up to 99.9% of the time goes to host loops that carry a value from one iteration to the next, the random-number generator that builds the graph in `wl_bfs` and `cc`, and a running maximum over the result in `bfs`. The whole-program interpreter runs all iterations of a loop at once only when they do not depend on each other, so it runs these loops one iteration at a time, at about a millisecond per iteration. The hybrid interpreter runs them natively and takes 2.0 and 2.9 seconds for the same two programs.

Under four thread orders. The extra thread orders are the main cost. Running all four costs the hybrid interpreter 5.1 to 370 times as much as lockstep alone, up to 1032 seconds for `rodinia_bfs` with 65,536 vertices, and the whole-program interpreter 4.1 to 323 times (1011 seconds for the same run). Only kernels that read and write the same unit in one launch are reordered (Section 3.8), but in the breadth-first searches and `cc` these are the main kernels, and in the reordered orders the interpreter carries out every statement once per warp of 32 threads instead of once per group of up to 2^{18} threads, so the cost grows with the number of warps. `wl_bfs`, whose launches have few threads, takes 46 seconds under all four orders in the hybrid interpreter, less than the whole-program interpreter's lockstep run. Both interpreters run the orders one after another.

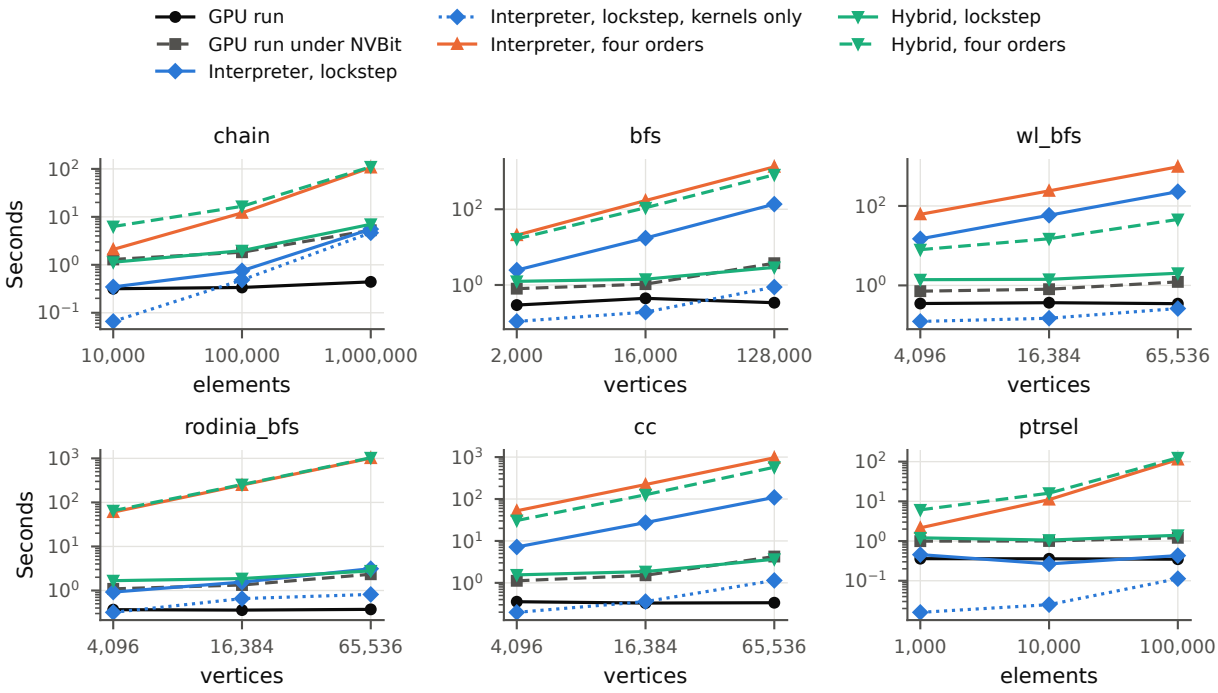


Figure 8: Wall-clock time against input size, both axes logarithmic: the program on the GPU (CUDA start-up included), the program under the NVBit tool, the whole-program interpreter in lockstep, the part of that time spent running kernels, and under all four thread orders together, and the hybrid interpreter in lockstep and under all four orders.

After the lockstep run the other three are independent of each other, so they could run at the same time on separate cores, and a cheaper way to sample orders, such as reordering only the warps whose units overlap, would reduce their cost further.

5.8 Before the input is known

The interpreter needs the input. To see what depends on the input and what does not, we ran `rodinia_bfs` on three random graphs (Table 9). The searches took 51 to 63 operations and produced 417 to 656 dependences, and only 48 dependences have the same operations and units in all three graphs, most of them through the one-unit flag that the host copies back after every round. The exact bytes are therefore a property of the input. We then describe each dependence by a *pattern*: the names of its two operations, its kind, its buffer, and how many operations apart they are, for example "`expand` then `commit` one operation later, RAW through `updating`". All 24 patterns of the needed dependences are the same for the three graphs. The structure of the dependences is therefore a property of the program, and only the number of rounds and the units are a property of the input. A tool that must decide before the input is known could use the patterns, and refine them with the interpreter once the input is available. We have not built such a tool. Run without the input, the interpreter today stops after one round of the search, because the loop's exit condition comes from the GPU and is unknown, and it reports 15 of the 22 patterns that connect operations at most six apart, nearly all of them as possible.

				GPU (s)		Whole program			Hybrid	
Program	Input	GPU threads	Operations	Run	Under NVBit	Lockstep (s)	In kernels	Four orders (s)	Lockstep (s)	Four orders (s)
`chain`	10,000	30,592	5	0.32	1.29	0.35	19%	2.1	1.13	6.3
`chain`	100,000	300,544	5	0.34	1.83	0.75	64%	12.1	1.97	16.6
`chain`	1,000,000	3,000,448	5	0.44	5.26	5.55	84%	106.6	7.00	110.4
`bfs`	2,000	30,720	49	0.29	0.80	2.47	4%	20.6	1.25	16.5
`bfs`	16,000	288,000	58	0.44	1.05	17.33	1%	169.8	1.42	109.5
`bfs`	128,000	2,816,000	70	0.34	3.76	136.50	1%	1319.2	2.92	814.4
`rodinia_bfs`	4,096	106,496	59	0.37	1.09	0.92	35%	60.3	1.67	64.9
`rodinia_bfs`	16,384	524,288	71	0.36	1.33	1.55	42%	247.6	1.87	254.5
`rodinia_bfs`	65,536	2,228,224	75	0.37	2.34	3.13	26%	1011.0	2.79	1032.4
`wl_bfs`	4,096	5,120	56	0.35	0.72	14.87	1%	61.5	1.39	7.9
`wl_bfs`	16,384	16,128	59	0.37	0.80	58.31	0%	239.6	1.42	14.9
`wl_bfs`	65,536	60,288	71	0.35	1.23	229.03	0%	965.7	2.02	45.7
`cc`	4,096	102,400	76	0.35	1.12	7.17	3%	52.3	1.55	30.7
`cc`	16,384	409,600	76	0.33	1.52	27.41	1%	221.4	1.86	126.3
`cc`	65,536	1,835,008	85	0.34	4.24	109.34	1%	963.6	3.66	572.7
`ptrsel`	1,000	5,120	10	0.36	0.99	0.46	4%	2.1	1.20	6.1
`ptrsel`	10,000	50,560	10	0.36	1.00	0.27	9%	11.0	1.06	16.1
`ptrsel`	100,000	500,480	10	0.35	1.21	0.43	26%	112.6	1.40	125.1

Table 8: Run times. GPU threads is the total number of threads of all launches. The GPU columns are wall-clock times of the whole program, CUDA start-up included (median of three runs). "In kernels" is the share of the lockstep whole-program interpreter's time spent running kernels; the rest runs host code. "Four orders" is the total time of the runs under all four thread orders.

Input	Vertices	Edges	Operations	Dependences	Needed	Same bytes in all graphs	Needed patterns in all graphs
graph A	4,096	12,184	59	610	155	48	24/24
graph B	4,096	12,289	63	656	168	48	24/24
graph C	8,192	41,222	51	417	129	48	24/24
no input	–	–	11	56	37	–	15/22

Table 9: `rodinia_bfs` on three graphs. "Same bytes in all graphs" counts dependences with the same operations, kind, buffer and unit count in all three. A pattern is a dependence without its position: the names of its operations, kind, buffer and distance in operations.

6 Related work

Static analysis of GPU kernels. Compilers determine the dependences of loops and kernels whose addresses are affine functions of loop indices with the polyhedral model [10, 12], but an address read from memory falls outside that model. GPUVerify [4] proves that a kernel is free of data races for all inputs, and GKLEE [14] explores a kernel's behaviour with symbolic inputs. Both reason about one kernel and treat memory contents as arbitrary, so neither tells which bytes a kernel touches on a given input, and neither relates one launch to the next. Our interpreters answer the latter question, for one input.

Dynamic analysis and emulation. NVBit [20] and race detectors such as BARRACUDA [9] observe the accesses of a running program on the GPU, as our ground-truth tool does. They give the exact bytes of one run, but only by running the program on the GPU with instrumentation, and only after the fact. GPU emulators and simulators such as Ocelot [8] and GPGPU-Sim [3] run compiled kernels on the CPU, one thread or warp after another, and could record the same bytes for one order of execution. Our interpreters run kernels from source with all threads as arrays, report which dependences are certain and which only possible, and cover other orders than the one they ran.

Inspector and executor. Runtime parallelisation of irregular loops runs an *inspector* that reads the index arrays to compute the loop's dependences, and then an *executor* that runs the loop in an order that respects them [19]. The interpreters are inspectors for a whole CUDA program. For accesses whose indices come only from the input, an inspector can be cheap, but in the breadth-first searches the indices depend on values earlier kernels computed, so finding the dependences means computing those values, which the interpreters do.

Dependence-driven GPU scheduling. Wireframe [2] and BlockMaestro [1] run GPU work from a dependence graph between thread blocks of consecutive kernels, and CUDA graphs [18] let the programmer state dependences between operations. They need the dependences as input. BlockMaestro derives them from the kernels' memory accesses when these are regular; the interpreters could supply them for irregular kernels when the input is known.

7 Limitations

A limitation of the interpreters is that they need the program's input, which means that their results are for that input only (Section 5.8). A tool that schedules from them must run them on the input it will use.

A limitation of the whole-program interpreter is that it must understand all of the host code, which means that programs whose host code uses C++ streams, option parsers or old C dialects come out with operations missing (Section 5.4). It warns when this happens; the hybrid interpreter does not have this limitation.

A limitation of the over-approximation for thread order is that it reports every dependence another order could produce, which means that kernels whose result depends on thread order come out with many possible dependences that a given run does not have (1,443 over the benchmarks). It also rests on the units the sampled orders touched: a GPU run that touched a unit none of the four orders touched could still produce a dependence that is not reported, although none of our traced runs did.

A limitation of splitting threads is that it applies only to a pointer loaded before any store in its statement, which means that a pointer loaded after an atomic operation in the same statement still makes every access through it a possible access to every buffer.

A limitation of the cost is that the interpreters simulate every GPU thread on one CPU core in Python, which means that their run time grows with the number of threads, and that GraphBIG's `betweenness`, ten times larger than any other benchmark, did not finish in two hours. The whole-program interpreter's run time also grows with sequential host loops, and the extra thread orders multiply the cost of either interpreter by up to 370 for programs whose reordered kernels have many warps (Section 5.7).

A limitation of the evaluation is that every program was traced once on one GPU, with inputs small enough for the tracer, which means that the scores are for those runs and inputs.

8 Conclusion

We showed that the memory dependences between the operations of a CUDA program, including those that pass through bytes chosen by data, can be found before the program runs on the GPU, by running its kernels in an interpreter on the CPU with its input. Running all threads of a kernel at once as arrays keeps the cost proportional to the work of the kernels, splitting threads by the buffer their pointers name makes accesses through pointer tables exact, and running several legal thread orders, with an over-approximation of what other orders could do, keeps every dependence a GPU run has among the reported ones. Running the host code natively removes the need to understand it. On 37 unmodified benchmarks, the hybrid interpreter missed no dependence and reported no wrong certain dependence. The next steps are to reduce the possible dependences reported for kernels whose result depends on thread order, and to reduce the cost for programs with very many launches.

References

[1] AmirAli Abdolrashidi, Hoda Aghaei Esfeden, Ali Jahanshahi, Kaustubh Singh, Nael Abu-Ghazaleh, and Daniel Wong. BlockMaestro: Enabling programmer-transparent task-based execution in GPU systems. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 333–346, 2021.

[2] AmirAli Abdolrashidi, Devashree Tripathy, Mehmet Esat Belviranli, Laxmi Narayan Bhuyan, and Daniel Wong. Wireframe: Supporting data-dependent parallelism through dependency graph execution in GPUs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 600–611, 2017.

[3] Ali Bakhoda, George L. Yuan, Wilson W. L. Fung, Henry Wong, and Tor M. Aamodt. Analyzing CUDA workloads using a detailed GPU simulator. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 163–174, 2009.

[4] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. GPUVerify: A verifier for GPU kernels. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 113–132, 2012.

[5] Martin Burtscher, Rupesh Nasre, and Keshav Pingali. A quantitative study of irregular programs on GPUs. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–151, 2012.

[6] Shuai Che, Bradford M. Beckmann, Steven K. Reinhardt, and Kevin Skadron. Pannotia: Understanding irregular GPGPU graph applications. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 185–195, 2013.

[7] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54, 2009.

[8] Gregory Diamos, Andrew Kerr, Sudhakar Yalamanchili, and Nathan Clark. Ocelot: A dynamic optimization framework for bulk-synchronous applications in heterogeneous systems. In *Proceedings of the 19th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 353–364, 2010.

[9] Ariel Eizenberg, Yuanfeng Peng, Toma Pigli, William Mansky, and Joseph Devietti. BARRACUDA: Binary-level analysis of runtime RAces in CUDA programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 126–140, 2017.

[10] Paul Feautrier. Dataflow analysis of array and scalar references. *International Journal of Parallel Programming*, 20(1):23–53, 1991.

[11] Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. Auto-tuning a high-level language targeted to GPU codes. In *Proceedings of Innovative Parallel Computing (InPar)*, 2012.

[12] Tobias Grosser, Armin Größlinger, and Christian Lengauer. Polly — performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters*, 22(4), 2012.

[13] International Organization for Standardization. *Programming Languages — C++, ISO/IEC 14882:2017*, 2017. Section 8.2.1 [expr.sub]: the expression $E1[E2]$ is sequenced with $E1$ before $E2$.

[14] Guodong Li, Peng Li, Geof Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. GKLEE: Concolic verification and test generation for GPUs. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 215–224, 2012.

[15] The LLVM Project. *libclang: C Interface to Clang.* `https://clang.llvm.org/doxygen/group__CINDEX.html`.

[16] Nafis Mustakin. From source code to sampling correction: Predicting memory-footprint models for unseen programs. Technical report, University of California, Riverside, 2026. MemPrint report, `paper_generalization/` in the MemPrint repository.

[17] Lifeng Nai, Yinglong Xia, Ilie Gabriel Tanase, Hyesoon Kim, and Ching-Yung Lin. GraphBIG: Understanding graph computing in the context of industrial solutions. In *Proceedings of the*

International Conference for High Performance Computing, Networking, Storage and Analysis (SC), 2015.

[18] NVIDIA Corporation. *CUDA C++ Programming Guide, Release 13*, 2026. Sections on streams and CUDA graphs.

[19] Joel H. Saltz, Ravi Mirchandaney, and Kay Crowley. Run-time parallelization and scheduling of loops. *IEEE Transactions on Computers*, 40(5):603–612, 1991.

[20] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 372–383, 2019.

[21] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2016.